

SAVITRIBAI PHULE PUNE UNIVERSITY

Third Year Computer Engineering (2019 Course)

PRACTICAL EXAM THEORY NOTES

Subject 1: Cloud Computing
Subject 2: Artificial Intelligence

Assignments 1–5 | Cloud Computing
Assignments 1–6 | Artificial Intelligence (Group A, B, C)

Includes: Definitions • Use Cases • Architecture •
Time & Space Complexity • Key Points • Exam Tips

Academic Year 2025–26

Assignment 1A: Case Study on Microsoft Azure

1. Definition

Microsoft Azure is a comprehensive cloud computing platform and infrastructure created by Microsoft. It provides a wide range of cloud services including computing, analytics, storage, and networking. Users can pick and choose from these services to develop and scale new applications, or run existing applications in the public cloud. Azure operates through a global network of Microsoft-managed data centers.

2. Key Features & Architecture

- IaaS (Infrastructure as a Service): Virtual Machines, Virtual Networks, Storage Accounts
- PaaS (Platform as a Service): Azure App Service, Azure SQL Database, Azure Functions
- SaaS (Software as a Service): Microsoft 365, Dynamics 365
- Global Regions: 60+ regions worldwide including Central India, South India
- Availability Zones: Physically separate locations within an Azure region for high availability
- Azure Resource Manager (ARM): Deployment and management service for Azure resources

3. Core Services

- Azure Virtual Machines: On-demand scalable computing resources
- Azure Blob Storage: Massively scalable object storage for unstructured data
- Azure SQL Database: Managed relational database service
- Azure Active Directory: Cloud-based identity and access management
- Azure Kubernetes Service (AKS): Managed container orchestration
- Azure DevOps: Development collaboration tools for CI/CD pipelines
- Azure IoT Hub: Managed service for IoT device connectivity

4. Use Cases

- Web Application Hosting: Deploy scalable web apps without managing servers
- Big Data Analytics: Process massive datasets using Azure HDInsight and Synapse
- Disaster Recovery: Backup and replicate on-premises workloads to Azure
- AI & Machine Learning: Azure ML Studio for building and deploying ML models
- DevOps & CI/CD: Automate build, test, and deployment pipelines

5. Pricing Model

Azure uses a pay-as-you-go pricing model. You are charged only for what you use, with no upfront costs. Reserved instances (1 or 3 year) offer up to 72% discount. Azure Free Tier provides 12 months of popular free services + 55 services always free.

6. Advantages

- Scalability: Scale up/down resources instantly based on demand
- Reliability: 99.99% SLA for most services with redundancy built in
- Security: ISO 27001, SOC 1/2/3, HIPAA, PCI DSS certified
- Hybrid Cloud: Seamless integration with on-premises infrastructure via Azure Arc
- Cost Efficiency: No capital expenditure; operational expenditure model

7. Exam Key Points

- Azure was launched in February 2010 as 'Windows Azure', renamed in 2014
- Competes with AWS (Amazon) and GCP (Google)
- Azure holds ~23% of global cloud market share (2nd after AWS)
- Data centers run on renewable energy as part of Microsoft's carbon-neutral pledge

Assignment 1B: Case Study on Amazon EC2

1. Definition

Amazon Elastic Compute Cloud (Amazon EC2) is a web service that provides secure, resizable compute capacity in the cloud. It is designed to make web-scale cloud computing easier for developers. EC2 is part of Amazon Web Services (AWS) and allows users to rent virtual computers on which to run their own applications.

2. Key Concepts

- Instance: A virtual server in the AWS cloud (like a VM)
- AMI (Amazon Machine Image): Pre-configured template containing OS + software for instances
- Instance Types: Different CPU, memory, storage combinations (t2.micro, m5.large, c5.xlarge, etc.)
- Elastic IP: Static IPv4 address designed for dynamic cloud computing
- Security Groups: Virtual firewall controlling inbound/outbound traffic
- Key Pairs: Secure login using public-private key cryptography (SSH)
- EBS (Elastic Block Store): Persistent block storage volumes for EC2 instances

3. Instance Families

- General Purpose (t2, t3, m5): Balanced compute, memory, network — web servers, dev environments
- Compute Optimized (c5, c6): High-performance processors — batch processing, HPC
- Memory Optimized (r5, x1): Fast performance for large datasets in memory — databases
- Storage Optimized (i3, d2): High sequential read/write — data warehouses, Hadoop
- GPU Instances (p3, g4): Machine learning, graphics rendering

4. EC2 Features

- Auto Scaling: Automatically add/remove instances based on demand
- Load Balancing: Distribute traffic across multiple instances (ELB)
- Spot Instances: Use spare AWS capacity at up to 90% discount
- Reserved Instances: 1 or 3 year commitment for up to 75% savings
- Placement Groups: Influence placement of interdependent instances

5. Use Cases

- Web Application Servers: Host dynamic web applications globally
- High Performance Computing: Scientific simulations, genomics research
- Machine Learning: Train and deploy ML models on GPU instances
- Enterprise Applications: Run SAP, Oracle on scalable infrastructure
- Game Servers: Low latency game backend infrastructure

Metric	Value
Launch Time	Seconds to minutes
Availability SLA	99.99% for multi-AZ deployments
Storage (EBS)	Up to 64 TB per volume
Network Bandwidth	Up to 100 Gbps (some instance types)
Free Tier	750 hrs/month t2.micro for 12 months

6. Exam Key Points

- EC2 was launched in 2006 as one of AWS's first services
- 'Elastic' means you can grow or shrink capacity within minutes
- EC2 is IaaS — you manage OS, middleware, runtime, application
- Difference from Lambda: EC2 is always-on; Lambda is serverless/event-driven

Assignment 2: Installation & Configuration of Google App Engine

1. Definition

Google App Engine (GAE) is a fully managed Platform as a Service (PaaS) offered by Google Cloud Platform. It allows developers to build and deploy applications on Google's scalable infrastructure without managing the underlying servers, networking, or infrastructure. App Engine automatically handles scaling, load balancing, and server maintenance.

2. Types of Environments

- Standard Environment: Runs in a sandbox; supports Python, Java, Go, PHP, Node.js, Ruby; scales to zero when no traffic; instances start in seconds
- Flexible Environment: Runs in Docker containers on GCE VMs; supports any language; more control over runtime; minimum 1 instance always running

3. Key Components

- app.yaml: Configuration file defining runtime, scaling, URL routing, environment variables
- Cloud SDK (gcloud): Command-line tool for deploying and managing App Engine apps
- Cloud Console: Web-based GUI for monitoring, versioning, and traffic splitting
- Datastore / Cloud SQL: Managed databases for storing application data
- Cloud Storage: For storing static files, user uploads

4. Installation Steps

- Step 1: Create a Google Cloud account at console.cloud.google.com
- Step 2: Create a new project and note the Project ID
- Step 3: Enable App Engine API from APIs & Services
- Step 4: Install Google Cloud SDK: `curl https://sdk.cloud.google.com | bash`
- Step 5: Initialize SDK: `gcloud init` (login and select project)
- Step 6: Create app.yaml configuration file in project directory
- Step 7: Deploy application: `gcloud app deploy`
- Step 8: View live app: `gcloud app browse`

5. app.yaml Configuration

```
runtime: python39
env: standard
entrypoint: gunicorn -b :$PORT main:app
automatic_scaling:
min_instances: 0
max_instances: 10
```

6. Use Cases

- Web Application Hosting: REST APIs, web portals, e-commerce sites
- Mobile Backends: Backend APIs for Android/iOS apps
- Microservices: Deploy independent services with separate versions
- Scheduled Tasks: Cron jobs using Cloud Scheduler

Metric	Value
Type	PaaS (Platform as a Service)

Scaling	Automatic (0 to millions of instances)
Pricing	Pay per instance hour + resource usage
Free Tier	28 instance hours/day (Standard env)
Deployment Time	2–5 minutes
Supported Languages	Python, Java, Go, PHP, Node.js, Ruby, .NET

7. Advantages over Traditional Hosting

- No server management — Google handles patching, updates, scaling
- Built-in versioning — deploy new version, split traffic gradually
- Integrated with all GCP services — Cloud SQL, Pub/Sub, BigQuery
- Global CDN — serve static content from edge locations worldwide

Assignment 2B: Installation & Configuration of KVM Virtualization

1. Definition

KVM (Kernel-based Virtual Machine) is an open-source full virtualization solution built into the Linux kernel. It converts Linux into a Type-1 (bare-metal) hypervisor. KVM allows multiple virtual machines (VMs) running unmodified Linux or Windows images. Each VM has private virtualized hardware including CPU, memory, disk, and network.

2. Types of Virtualization

- Full Virtualization: Guest OS runs unmodified; hardware is emulated (KVM uses this)
- Para-virtualization: Guest OS is modified to communicate with hypervisor directly (Xen)
- OS-level Virtualization: Single OS kernel runs multiple isolated user-space instances (Docker)

3. KVM Architecture

- Host OS: Linux kernel with KVM module (/dev/kvm)
- QEMU: Hardware emulator that works with KVM for I/O virtualization
- libvirt: Management API/daemon (libvirtd) for controlling VMs
- virt-manager: GUI tool for creating and managing VMs
- virsh: Command-line tool for VM management

4. Installation Commands

```
# Check if CPU supports virtualization
egrep -c '(vmx|svm)' /proc/cpuinfo

# Install KVM and tools
sudo apt install qemu-kvm libvirt-daemon-system
sudo apt install libvirt-clients bridge-utils virt-manager

# Start and enable libvirt service
sudo systemctl start libvirtd
sudo systemctl enable libvirtd

# Add user to libvirt group
sudo usermod -aG libvirt $USER

# Launch GUI
sudo virt-manager
```

5. Key Commands (virsh)

```
virsh list --all # List all VMs
virsh start # Start a VM
virsh shutdown # Gracefully stop a VM
virsh snapshot-create-as snap1 # Create snapshot
```

6. Use Cases

- Server Consolidation: Run multiple server workloads on single physical machine
- Development/Testing: Isolated test environments without extra hardware
- Cloud Infrastructure: OpenStack uses KVM as default hypervisor
- Education Labs: Create multiple OS environments for student practice

Metric	Value
--------	-------

Type	Type-1 Hypervisor (bare-metal)
Performance Overhead	< 5% compared to bare metal
Max VMs per host	Depends on RAM/CPU (typically 10–100+)
Supported Guest OS	Linux, Windows, BSD, Solaris
License	Open Source (GPL)

Assignment 3: Creating an Application in Salesforce using Apex

1. Definition

Apex is a strongly typed, object-oriented programming language developed by Salesforce. It runs exclusively on the Force.com platform (Salesforce servers) and allows developers to execute flow and transaction control statements on Salesforce servers in conjunction with calls to the Force.com API. Apex syntax is similar to Java.

2. Key Characteristics of Apex

- Strongly Typed: All variables must be declared with a data type
- Object-Oriented: Supports classes, interfaces, inheritance, and polymorphism
- Cloud-Based: Runs entirely on Salesforce servers — no local installation needed
- Database Integration: SOQL (Salesforce Object Query Language) runs directly within Apex code
- Governor Limits: Apex enforces limits per transaction (100 SOQL queries, 150 DML statements)
- Multitenant: Shares resources with other Salesforce orgs; governor limits ensure fairness

3. Types of Apex

- Apex Classes: Reusable code modules with methods and properties
- Apex Triggers: Code that runs before/after DML events (insert, update, delete, undelete)
- Apex Test Classes: Mandatory unit tests (75% code coverage required for deployment)
- Visualforce Controllers: Server-side logic for Visualforce pages
- Batch Apex: Process large records asynchronously (up to 50M records)
- Scheduled Apex: Run code at scheduled times using Cron expressions

4. Development Steps

- Step 1: Create free Salesforce Developer org at developer.salesforce.com
- Step 2: Login and go to Setup (gear icon, top right)
- Step 3: Open Developer Console: Setup → Developer Console
- Step 4: File → New → Apex Class
- Step 5: Write class, Save (Ctrl+S), execute via Anonymous Window
- Step 6: Check output in Logs tab

5. Basic Apex Code Examples

```
// Apex Class
public class StudentManager {
    public static void createStudent(String name, Integer age) {
        Student__c s = new Student__c();
        s.Name = name;
        s.Age__c = age;
        insert s;
        System.debug('Student created: ' + name);
    }
}
```

```
// Apex Trigger
trigger StudentTrigger on Student__c (before insert) {
    for(Student__c s : Trigger.new) {
        if(s.Age__c < 18) {
```

```
s.addError('Age must be 18 or above');  
}  
}  
}
```

6. SOQL — Salesforce Query Language

```
// Query all students with marks > 60  
List students = [SELECT Name, Marks__c FROM Student__c WHERE Marks__c > 60];  
for(Student__c s : students) { System.debug(s.Name); }
```

7. Use Cases

- CRM Automation: Auto-create follow-up tasks when a deal is won
- Data Validation: Enforce business rules before data is saved
- Integration: Call external REST/SOAP APIs from Apex callouts
- Batch Processing: Process millions of records for data migration

Metric	Value
Governor Limit: SOQL queries	100 per transaction
Governor Limit: DML statements	150 per transaction
Governor Limit: CPU time	10,000 ms per transaction
Heap size	6 MB (sync), 12 MB (async)
Test coverage required	75% minimum for deployment

Assignment 4: Custom Application (Mini Project) using Salesforce Cloud

1. Definition

Salesforce Cloud is a CRM (Customer Relationship Management) platform that provides cloud-based tools for sales, service, marketing, and more. A custom application in Salesforce is built using point-and-click tools or code (Apex, LWC) to address specific business requirements beyond standard Salesforce functionality.

2. Key Building Blocks

- Custom Objects: User-defined tables (like database tables) — e.g., Student__c, Product__c
- Custom Fields: Additional columns added to standard or custom objects
- Relationships: Lookup (loose link) and Master-Detail (tight parent-child link)
- Lightning App: A branded application with a navigation bar and selected tabs/pages
- Lightning Pages: Drag-and-drop page layouts using App Builder
- Validation Rules: Formula-based rules that prevent incorrect data entry
- Workflow Rules / Flows: Automate actions based on record changes
- Reports & Dashboards: Visual analytics built from your data

3. Steps to Create a Custom App

- Step 1: Setup → Object Manager → Create → Custom Object
- Step 2: Name object (e.g., 'Student'), add fields (Name, Roll No, Marks, Department)
- Step 3: Setup → App Manager → New Lightning App
- Step 4: Name the app, choose icon, add tabs (your custom object)
- Step 5: Assign app to user profile (System Administrator)
- Step 6: Navigate to the app from App Launcher (9-dot grid)
- Step 7: Create records, set page layout, build reports

4. Lightning Web Components (LWC) Basics

```
{studentName}  
Marks: {marks}
```

```
// studentCard.js  
import { LightningElement, api } from 'lwc';  
export default class StudentCard extends LightningElement {  
  @api studentName;  
  @api marks;  
}
```

5. Use Cases

- Student Management System: Track student records, marks, attendance
- Inventory Tracker: Manage product stock, suppliers, orders
- Event Management: Register participants, track sessions, send confirmations
- HR Management: Employee records, leave tracking, performance reviews

6. Salesforce Cloud Types

- Sales Cloud: Lead, opportunity, account management for sales teams
- Service Cloud: Case management, knowledge base, omnichannel support

- Marketing Cloud: Email campaigns, customer journeys, social media
- Commerce Cloud: B2B and B2C e-commerce storefronts
- Health Cloud: Patient management for healthcare industry

Assignment 5: Mini Project — SaaS over LAN with HDFS & Encryption

1. Definition

Software as a Service (SaaS) is a cloud delivery model where software is hosted on remote servers and made available to users over the internet (or LAN) without local installation. HDFS (Hadoop Distributed File System) is a distributed file system designed to run on commodity hardware, providing high-throughput access to application data by splitting files into blocks and storing them across multiple nodes.

2. HDFS Core Concepts

- NameNode: Master server that manages file system metadata (directory tree, block locations)
- DataNode: Worker nodes that store actual data blocks
- Block Size: Default 128 MB (configurable); large files are split into blocks
- Replication Factor: Default 3 copies of each block for fault tolerance
- Rack Awareness: Replicas placed on different racks to survive rack failures
- Write-Once-Read-Many: HDFS is optimized for batch reads, not random writes

3. Cloud Controller Components

- File Splitter: Divides uploaded file into fixed-size blocks (simulating HDFS)
- Encryption Module: Encrypts each block using symmetric key (Fernet/AES)
- Block Storage: Saves encrypted blocks to storage nodes on LAN
- Metadata Store: Tracks block IDs, order, encryption key, file info
- Download Handler: Retrieves blocks in order, decrypts, and reassembles file

4. Technologies Used

- Python: Core programming language for cloud controller
- cryptography (Fernet): Symmetric encryption for file blocks
- Flask: Lightweight web framework to expose SaaS API over LAN
- Socket / HTTP: Communication between client and cloud controller
- File System: Local directories simulate distributed block storage

5. System Architecture

Client → [Upload File] → Cloud Controller

Cloud Controller → [Split into Blocks] → Block_0, Block_1, ..., Block_N

Cloud Controller → [Encrypt each Block] → Encrypted_Block_0 ... N

Cloud Controller → [Store Blocks] → /cloud_storage/ directory

Client → [Download Request] → Cloud Controller

Cloud Controller → [Fetch Blocks in Order] → Decrypt → Reassemble → File

6. Key Code Concepts

```
from cryptography.fernet import Fernet
key = Fernet.generate_key() # 256-bit symmetric key
cipher = Fernet(key)
encrypted = cipher.encrypt(data) # Encrypt
decrypted = cipher.decrypt(encrypted) # Decrypt
```

7. Complexity Analysis

Metric	Value
Upload Time Complexity	$O(N)$ — N = number of blocks
Download Time Complexity	$O(N)$ — reassemble N blocks
Space Complexity	$O(N)$ — N blocks stored on disk
Encryption Overhead	~5-10% overhead per block (Fernet)
Block Size (simulated)	1 KB – 1 MB (configurable)

8. Use Cases

- College Lab Cloud: Students upload/download assignments via LAN
- Hospital Records: Distributed encrypted storage of patient records
- Backup System: Automated backup with block-level deduplication
- Media Streaming: Split large video files, stream blocks on demand

9. Exam Key Points

- SaaS = No installation needed; accessed via browser over network
- HDFS block size default = 128 MB (Hadoop 2.x+)
- Fernet uses AES-128-CBC with HMAC-SHA256 for integrity
- Fault tolerance: if one block storage fails, retrieve from replica
- This assignment demonstrates IaaS + SaaS concepts together

GROUP A — Search Algorithms

Assignment 1: Depth First Search (DFS) & Breadth First Search (BFS)

1. Definition

DFS and BFS are fundamental graph/tree traversal algorithms used to visit all vertices of a graph or tree systematically. They differ in the order in which nodes are explored.

Depth First Search (DFS)

DFS explores as far as possible along each branch before backtracking. It uses a stack (or recursion) to keep track of the path. DFS is useful for detecting cycles, topological sorting, and solving maze problems.

- Strategy: Go deep first, backtrack when stuck
- Data Structure: Stack (implicit via recursion)
- Traversal Order: $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F$ (pre-order, left-to-right)

Breadth First Search (BFS)

BFS explores all neighbors of a node before moving to the next level. It uses a queue to process nodes level by level. BFS guarantees shortest path in unweighted graphs.

- Strategy: Explore all neighbors first (level by level)
- Data Structure: Queue (FIFO)
- Traversal Order: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$ (level order)

2. Time & Space Complexity

Metric	Value
DFS Time Complexity	$O(V + E)$ — V =vertices, E =edges
DFS Space Complexity	$O(V)$ — recursion stack depth
BFS Time Complexity	$O(V + E)$
BFS Space Complexity	$O(V)$ — queue stores all nodes at current level
Best for shortest path	BFS (guarantees shortest in unweighted graph)
Best for cycle detection	DFS
Best for memory	DFS (uses less memory than BFS for wide graphs)

3. Use Cases

- DFS: Maze solving, topological sort, finding connected components, detecting cycles
- BFS: Shortest path (unweighted), web crawlers, social network friend suggestions
- BFS: GPS navigation (fewest hops), peer-to-peer networks
- DFS: Solving puzzles (Sudoku, N-Queens), tree serialization

4. Key Differences

Metric	Value
Criterion	DFS vs BFS

Data Structure	Stack (DFS) vs Queue (BFS)
Memory Usage	Less (DFS) vs More (BFS)
Shortest Path	Not guaranteed (DFS) vs Guaranteed (BFS)
Implementation	Recursive or iterative (DFS) vs Iterative (BFS)
Completeness	Not always (DFS) vs Always (BFS)

5. Exam Key Points

- DFS can get trapped in infinite loops if visited set is not maintained
- BFS is always optimal for unweighted graphs — finds shortest path
- Both DFS and BFS work on disconnected graphs by iterating all vertices
- Recursive DFS may cause stack overflow for very deep graphs

Assignment 2: A* Algorithm for Game Search Problem

1. Definition

A* (A-Star) is an informed search algorithm that finds the shortest path between a start node and a goal node. It uses a heuristic function to estimate the cost from the current node to the goal, making it more efficient than uninformed searches. A* is widely used in games, robotics, and GPS navigation.

2. Core Formula

A* evaluates nodes using: $f(n) = g(n) + h(n)$

- $f(n)$: Total estimated cost of the cheapest solution through node n
- $g(n)$: Actual cost from start node to current node n
- $h(n)$: Heuristic — estimated cost from n to goal (must be admissible)
- Admissible Heuristic: Never overestimates actual cost ($h(n) \leq \text{true cost}$)

3. Common Heuristics

- Manhattan Distance: $|x1-x2| + |y1-y2|$ — for grid problems (8-puzzle, maze)
- Euclidean Distance: $\sqrt{(x1-x2)^2 + (y1-y2)^2}$ — for continuous spaces
- Hamming Distance: Number of misplaced tiles — for 8/15 puzzle
- Zero Heuristic: $h(n)=0$ makes A* behave like Dijkstra's algorithm

4. Algorithm Steps

- Step 1: Add start node to Open List (priority queue ordered by $f(n)$)
- Step 2: Pop node with lowest $f(n)$ from Open List
- Step 3: If it's the goal, reconstruct and return path
- Step 4: Move node to Closed List (visited)
- Step 5: For each neighbor, calculate g , h , f values
- Step 6: If neighbor not in Closed List and better path found, add to Open List
- Step 7: Repeat from Step 2 until goal found or Open List empty

5. Application: 8-Puzzle Problem

The 8-puzzle has a 3x3 grid with tiles 1-8 and one blank space. The goal is to reach the target configuration by sliding tiles. A* with Manhattan distance heuristic solves it optimally.

- State: Current arrangement of tiles (3x3 tuple)
- Actions: Slide blank up/down/left/right
- Goal Test: State matches goal configuration
- Heuristic: Sum of Manhattan distances of each tile from its goal position

6. Time & Space Complexity

Metric	Value
Time Complexity	$O(b^d)$ — b =branching factor, d =depth of solution
Space Complexity	$O(b^d)$ — stores all generated nodes
Optimal?	Yes, if heuristic is admissible
Complete?	Yes, if solution exists and branching factor is finite
Best Case	$O(d)$ — if heuristic perfectly estimates true cost

Comparison with BFS	Much faster due to heuristic guidance
---------------------	---------------------------------------

7. Use Cases

- Game AI: Pathfinding in video games (Pac-Man, strategy games)
- Robotics: Robot navigation around obstacles
- GPS Navigation: Finding shortest driving route
- Puzzle Solving: 8-puzzle, 15-puzzle, Rubik's cube
- Network Routing: Finding optimal data packet path

Assignment 3: Greedy Search Algorithms

1. Definition

A Greedy Algorithm makes locally optimal choices at each step with the hope of finding a global optimum. It never reconsiders its choices — once a decision is made, it is final. Greedy algorithms are fast but may not always produce the globally optimal solution.

I. Selection Sort

Repeatedly finds the minimum element from the unsorted portion and places it at the beginning.

Metric	Value
Time Complexity (Best)	$O(n^2)$
Time Complexity (Worst)	$O(n^2)$
Space Complexity	$O(1)$ — in-place sorting
Stable?	No (default), can be made stable
Use Case	Small datasets, when memory is limited

II. Minimum Spanning Tree (Generic)

A Minimum Spanning Tree (MST) of a weighted undirected graph connects all vertices with the minimum possible total edge weight, containing exactly $V-1$ edges and no cycles.

- Applications: Network design, cable laying, road construction, circuit design

III. Single-Source Shortest Path (Bellman-Ford)

Finds shortest path from one source vertex to all other vertices. Unlike Dijkstra, Bellman-Ford handles negative weight edges and detects negative cycles.

Metric	Value
Time Complexity	$O(V * E)$ — V =vertices, E =edges
Space Complexity	$O(V)$
Handles Negative Weights?	Yes
Detects Negative Cycles?	Yes

IV. Job Scheduling Problem

Given N jobs with deadlines and profits, schedule jobs to maximize total profit. Each job takes unit time. A job can only be scheduled if its deadline hasn't passed.

- Greedy Strategy: Sort by profit (descending), assign to latest available slot before deadline

Metric	Value
Time Complexity	$O(N^2)$ — naive; $O(N \log N)$ with disjoint sets
Space Complexity	$O(N)$
Optimal?	Yes — greedy works for this problem
Use Case	CPU scheduling, task management, resource allocation

V. Prim's MST Algorithm

Builds MST by starting from any vertex and greedily adding the minimum weight edge that connects the growing MST to a new vertex. Uses a min-heap/priority queue.

Metric	Value
Time Complexity	$O(E \log V)$ with binary heap
Space Complexity	$O(V + E)$
Best for	Dense graphs (many edges)
Approach	Vertex-based — grows the MST vertex by vertex

VI. Kruskal's MST Algorithm

Builds MST by sorting all edges by weight and adding them one by one, skipping edges that would form a cycle. Uses Union-Find (Disjoint Set Union) data structure.

Metric	Value
Time Complexity	$O(E \log E)$ — dominated by sorting
Space Complexity	$O(V + E)$
Best for	Sparse graphs (fewer edges)
Approach	Edge-based — processes edges in sorted order
Cycle Detection	Union-Find with path compression

VII. Dijkstra's Shortest Path Algorithm

Finds shortest path from source to all vertices in a weighted graph with non-negative weights. Uses a greedy approach with a min-heap priority queue.

Metric	Value
Time Complexity	$O((V + E) \log V)$ with binary heap
Space Complexity	$O(V)$
Handles Negative Weights?	No — use Bellman-Ford instead
Optimal?	Yes, for non-negative weights
Use Case	GPS navigation, network routing (OSPF protocol)

Key Difference: Prim's vs Kruskal's vs Dijkstra's

Metric	Value
Algorithm	Purpose / Key Difference
Prim's	MST — grows from a vertex, picks min edge to new vertex
Kruskal's	MST — sorts all edges, uses Union-Find to avoid cycles
Dijkstra's	Shortest Path — not MST; finds min dist from source to all nodes

Assignment 4: N-Queens Problem — Backtracking & Branch and Bound

1. Definition

The N-Queens problem requires placing N chess queens on an NxN chessboard such that no two queens attack each other. Queens attack horizontally, vertically, and diagonally. It is a classic Constraint Satisfaction Problem (CSP) solved using backtracking.

2. Constraint Satisfaction Problem (CSP)

- Variables: Position of each queen (one per row)
- Domain: Column positions {1, 2, ..., N} for each queen
- Constraints: No two queens share same row, column, or diagonal
- Goal: Find assignment satisfying all constraints simultaneously

3. Backtracking Algorithm

Backtracking incrementally builds candidates for the solution and abandons (backtracks) a candidate as soon as it determines it cannot lead to a valid solution.

- Step 1: Place queen in first column of current row
- Step 2: Check if placement is safe (no conflicts)
- Step 3: If safe, recurse to next row
- Step 4: If not safe, try next column
- Step 5: If all columns tried and none safe, backtrack to previous row
- Step 6: If all N queens placed, record solution

4. Branch and Bound

Branch and Bound is an optimization of backtracking that prunes branches early. It uses a bounding function to determine if a partial solution can lead to a better solution than the best found so far. For N-Queens, it tracks attacked columns and diagonals.

- Branching: Try each column position for current row queen
- Bounding: Skip columns already attacked by previous queens
- Pruning: Eliminate branches where constraints are already violated

5. Safety Check Logic

```
def is_safe(board, row, col):  
    for r in range(row):  
        # Same column attack  
        if board[r] == col: return False  
        # Diagonal attack  
        if abs(board[r] - col) == abs(r - row): return False  
    return True
```

6. Time & Space Complexity

Metric	Value
Time Complexity	$O(N!)$ — worst case tries all permutations

Space Complexity	$O(N)$ — recursion stack depth
Solutions for $N=4$	2 solutions
Solutions for $N=8$	92 solutions
Solutions for $N=12$	14,200 solutions
Pruning Effect	Reduces actual nodes explored significantly

7. Use Cases

- Constraint Satisfaction: Scheduling, resource allocation, timetabling
- VLSI Design: Circuit layout without signal interference
- Parallel Processing: Task assignment without resource conflicts
- AI Planning: Automated reasoning under constraints

Assignment 5: Elementary Chatbot for Customer Interaction

1. Definition

A chatbot is a software application designed to simulate human conversation through text or voice interactions. An elementary rule-based chatbot uses pattern matching (regular expressions) or keyword detection to map user inputs to predefined responses. More advanced chatbots use NLP (Natural Language Processing) and ML models.

2. Types of Chatbots

- Rule-Based (Pattern Matching): Uses if-else or regex rules; deterministic; no learning
- Retrieval-Based: Selects response from predefined set using ML similarity scoring
- Generative: Uses deep learning (GPT, BERT) to generate novel responses
- Hybrid: Combines rule-based for structured queries + ML for open-ended chat

3. Components of a Chatbot

- NLU (Natural Language Understanding): Extracts intent and entities from user input
- Dialogue Manager: Tracks conversation state and determines next action
- NLG (Natural Language Generation): Formulates the response in natural language
- Knowledge Base: Database of facts, FAQs, product info the bot can access
- Intent: What the user wants (e.g., 'check order status', 'get refund')
- Entity: Key information in the query (e.g., order ID, product name, date)

4. Pattern Matching with Regex

```
import re
pattern = r'order|track|status'
if re.search(pattern, user_input.lower()):
    response = 'Please share your order ID.'
```

5. Use Cases

- Customer Support: Answer FAQs 24/7 without human agents
- E-commerce: Track orders, process returns, recommend products
- Banking: Check balance, transaction history, report lost cards
- Healthcare: Symptom checker, appointment booking, medication reminders
- HR: Leave applications, policy queries, onboarding assistance

6. Complexity

Metric	Value
Pattern Matching Time	$O(P * M)$ — P=patterns, M=message length
Space Complexity	$O(P)$ — store P patterns
Response Time	< 100ms for rule-based systems
Accuracy (rule-based)	High for known patterns; 0% for unknown
Scalability	Poor for complex conversations without ML

7. Limitations of Rule-Based Chatbots

- Cannot handle misspellings or synonyms unless explicitly programmed

- No learning — must manually update rules for new topics
- Context-unaware — each message treated independently (no memory)
- Fails on complex multi-turn conversations requiring context tracking

Assignment 6: Expert Systems — All Subtypes

1. Definition

An Expert System is an AI program that emulates the decision-making ability of a human expert in a specific domain. It uses a knowledge base of facts and rules, combined with an inference engine, to reason and provide expert-level solutions to complex problems.

2. Components of an Expert System

- Knowledge Base: Stores domain-specific facts and rules (IF-THEN rules)
- Inference Engine: Applies logical rules to the knowledge base to derive conclusions
- Working Memory: Stores current facts and intermediate results during reasoning
- User Interface: Allows non-experts to interact with the system (question/answer)
- Explanation Module: Explains HOW and WHY the system reached its conclusion
- Knowledge Acquisition Module: Interface for experts to update the knowledge base

3. Inference Methods

- Forward Chaining: Start from known facts, apply rules, derive new facts (data-driven)
- Backward Chaining: Start from goal, work backwards to find supporting facts (goal-driven)
- Fuzzy Logic: Handle uncertainty and partial truths (not just true/false)

I. Information Management Expert System

Recommends optimal data storage and management solutions based on data size, access frequency, security requirements, and real-time processing needs.

- Rules: Large + Real-time → Hadoop + Kafka; Medium + Frequent → MySQL + Redis
- Use Case: IT departments choosing database/storage architecture
- Benefit: Reduces need for specialized database architects for routine decisions

II. Hospital & Medical Expert System

Assists doctors in diagnosing diseases based on patient symptoms. Uses IF-THEN rules mapping symptom combinations to probable diagnoses and treatment recommendations.

- Example Rule: IF fever AND cough AND breathlessness THEN possible COVID/Pneumonia
- Use Cases: MYCIN (1970s) — bacterial infection diagnosis; CADUCEUS — internal medicine
- Benefit: 24/7 availability, consistent diagnosis, second opinion for doctors
- Limitation: Cannot replace clinical judgment; liability issues for wrong diagnosis

III. Help Desk Management Expert System

Automatically categorizes IT support tickets, suggests solutions, and routes issues to the appropriate team based on category, severity, and past resolution history.

- Rule: IF category=network AND severity=high THEN escalate to Network Team, SLA=2hrs
- Use Case: IT helpdesks, customer service centers, telecom support
- Benefit: Faster resolution, consistent SLA adherence, reduced tier-1 support load

IV. Employee Performance Evaluation Expert System

Evaluates employee performance based on quantifiable metrics and recommends HR actions such as promotion, increment, training, or warning based on weighted scoring.

- Metrics: Attendance %, Targets Achieved %, Teamwork, Punctuality, Training completion
- Scoring: Each metric assigned weight; total score determines rating and recommendation
- Use Case: Annual appraisal automation, bias-free evaluation, HR analytics

V. Stock Market Trading Expert System

Analyzes stock indicators and market sentiment to provide buy/hold/sell recommendations. Uses technical analysis rules based on RSI, P/E ratio, volume, and trend direction.

- RSI > 70: Overbought → SELL signal
- RSI < 30: Oversold → BUY signal
- P/E > 40: Overvalued → HOLD/SELL
- Use Case: Retail investor guidance, algorithmic trading triggers, portfolio management
- Limitation: Cannot predict black swan events; past patterns may not repeat

VI. Airline Scheduling Expert System

Determines flight go/no-go decisions and scheduling actions based on weather, crew availability, aircraft serviceability, fuel status, and passenger load.

- Rule: IF weather=stormy THEN CANCEL and notify passengers
- Rule: IF passengers > capacity THEN overbooking protocol — bump volunteers
- Use Case: Airline operations control centers, ground staff decision support
- Benefit: Real-time decisions, safety compliance, reduced human error

4. General Complexity of Expert Systems

Metric	Value
Rule Matching Time	$O(R * F)$ — R=rules, F=facts in working memory
Space Complexity	$O(R + F)$ — store rules and facts
Inference Cycles	Depends on rule chaining depth
Knowledge Acquisition	Bottleneck — requires domain expert time
Accuracy	High within domain; fails outside knowledge base

5. Advantages of Expert Systems

- Available 24/7 without fatigue or emotional bias
- Consistent decisions — same inputs always produce same outputs
- Captures and preserves expert knowledge for future use
- Can explain reasoning (transparency) unlike black-box ML models
- Cost-effective for routine expert consultations

6. Limitations of Expert Systems

- Knowledge Acquisition Bottleneck: Hard and expensive to extract expert knowledge
- Domain Specific: Cannot transfer knowledge between different domains
- No Learning: Rule-based systems don't improve from experience
- Cannot Handle Uncertainty: Pure rule systems struggle with ambiguous inputs
- Maintenance: Rules must be manually updated as domain knowledge evolves

7. Famous Expert Systems in History

- MYCIN (1976): Diagnosed blood infections and recommended antibiotics — Stanford
- DENDRAL (1965): Identified chemical compounds from mass spectrometry data

- XCON (1980): Configured DEC VAX computer systems — saved \$40M/year
 - CADUCEUS: Diagnosis of internal medicine diseases
-

END OF THEORY NOTES
Best of Luck for Your Practical Exam!